Module III — Printable Code & Outputs Packet

Q9 / Q11 supporting material

Mohammed Ashar Husain | Roll No. 2024bcs0137

Current MPI implementation, shared graph header, captured MPI results, current comparison summary, and graph.

This packet reproduces current project artifacts only. It does not claim historical Assignment 1 source or handwritten answers.

Q9 — Current MPI source: c/mpi_pagerank.c

Mohammed Ashar Husain | Roll No. 2024bcs0137

```
#include "shared_graph.h"

#include <mpi.h>

int main(int argc, char **argv)
{
    int rank_id, world_size;
    PRCase c;
    PRGraph graph = {0};
    double *rank = NULL, *next = NULL, *local_values = NULL;
    double final_error = 0.0;
    int *counts = NULL, *displacements = NULL;
    size_t begin, end, local_n, iterations = 0;
    int success = 0;

    if (MPI_Init(&argc, &argv) != MPI_SUCCESS)
        return EXIT_FAILURE;
    if (MPI_Comm_rank(MPI_COMM_WORLD, &rank_id) != MPI_SUCCESS ||
        MPI_Comm_size(MPI_COMM_WORLD, &world_size) != MPI_SUCCESS)
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    if (!pr_parse_case(argc, argv, &c)) {
        if (rank_id == 0) pr_usage(argv[0]);
        MPI_Finalize();
        return EXIT_FAILURE;
    }
    /* Replicated deterministic graph: each rank stores the same incoming CSR,
     * and owns only a contiguous range of vertices for each update. */
    if (!pr_graph_build(&graph, &c) || !pr_graph_valid(&graph)) {
        if (rank_id == 0) fprintf(stderr, "Unable to construct graph\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    if (world_size <= 0 || graph.n > (size_t)INT_MAX) {
        if (rank_id == 0) fprintf(stderr, "Graph partition exceeds MPI count limits\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    begin = graph.n * (size_t)rank_id / (size_t)world_size;
    end = graph.n * (size_t)(rank_id + 1) / (size_t)world_size;
    local_n = end - begin;
    rank = pr_initial_ranks(graph.n);
    next = pr_rank_vector(graph.n);
    local_values = local_n > SIZE_MAX / sizeof(*local_values) ? NULL :
        (double *)malloc((local_n == 0 ? 1 : local_n) * sizeof(*local_values));
    if ((size_t)world_size > SIZE_MAX / sizeof(*counts) ||
        (size_t)world_size > SIZE_MAX / sizeof(*displacements))
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    counts = (int *)malloc((size_t)world_size * sizeof(*counts));
    displacements = (int *)malloc((size_t)world_size * sizeof(*displacements));
    if (rank == NULL || next == NULL || local_values == NULL || counts == NULL || displacements == NULL) {
        fprintf(stderr, "Rank %d: unable to allocate rank vectors\n", rank_id);
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    for (int p = 0; p < world_size; ++p) {
        const size_t first = graph.n * (size_t)p / (size_t)world_size;
        const size_t last = graph.n * (size_t)(p + 1) / (size_t)world_size;
        if (last - first > (size_t)INT_MAX || first > (size_t)INT_MAX) {
            if (rank_id == 0) fprintf(stderr, "MPI count/displacement overflow\n");
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        }
        counts[p] = (int)(last - first);
        displacements[p] = (int)first;
    }

    if (MPI_Barrier(MPI_COMM_WORLD) != MPI_SUCCESS)
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    const double start = MPI_Wtime();
    for (size_t iter = 1; iter <= PR_MAX_ITERATIONS; ++iter) {
        double local_dangling = 0.0, dangling = 0.0;
        double local_error = 0.0, error = 0.0;
        for (size_t v = begin; v < end; ++v)
            if (graph.out_degree[v] == 0) local_dangling += rank[v];
        /* Collective order is identical on all ranks: dangling sum, rank
         * all-gather, then global L1 sum. */
        if (MPI_Allreduce(&local_dangling, &dangling, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD) != MPI_SUCCESS)
```

```
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        const double base = (1.0 - PR_DAMPING) / (double)graph.n +
                            PR_DAMPING * dangling / (double)graph.n;
        for (size_t v = begin; v < end; ++v) {
            double incoming = 0.0;
            for (size_t p = graph.in_offsets[v]; p < graph.in_offsets[v + 1]; ++p) {
                const size_t src = (size_t)graph.in_sources[p];
                incoming += rank[src] / (double)graph.out_degree[src];
            }
            local_values[v - begin] = base + PR_DAMPING * incoming;
        }
        /* Gather into a distinct receive vector (no overlapping MPI buffers),
         * making all remote new ranks available for the next iteration. */
        if (MPI_Allgatherv(local_values, (int)local_n, MPI_DOUBLE, next, counts,
                           displacements, MPI_DOUBLE, MPI_COMM_WORLD) != MPI_SUCCESS)
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        for (size_t v = begin; v < end; ++v)
            local_error += next[v] >= rank[v] ? next[v] - rank[v] : rank[v] - next[v];
        if (MPI_Allreduce(&local_error, &error, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD) != MPI_SUCCESS)
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        final_error = error;
        { double *tmp = rank; rank = next; next = tmp; }
        iterations = iter;
        if (error < PR_TOLERANCE) { success = 1; break; }
    }
    {
        const double local_elapsed = MPI_Wtime() - start;
        double elapsed = 0.0;
        if (MPI_Reduce(&local_elapsed, &elapsed, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD) != MPI_SUCCESS)
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        if (rank_id == 0)
            pr_print_result(&graph, iterations, elapsed, success, final_error,
                            c.emit_ranks, rank);
    }
    free(rank); free(next); free(local_values); free(counts); free(displacements);
    pr_graph_free(&graph);
    MPI_Finalize();
    return success ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Q9 — Shared implementation header: c/shared_graph.h

Mohammed Ashar Husain | Roll No. 2024bcs0137

```
#ifndef PAGERANK_SHARED_GRAPH_H
#define PAGERANK_SHARED_GRAPH_H

#include <stddef.h>
#include <stdint.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define PR_DAMPING 0.85
#define PR_TOLERANCE 1e-6
#ifndef PR_MAX_ITERATIONS
#define PR_MAX_ITERATIONS 10000
#endif

static uint64_t pr_mul_mod(uint64_t a, uint64_t b, uint64_t mod)
{
    uint64_t result = 0;
    a %= mod;
    while (b != 0) {
        if (b & 1U)
            result = result >= mod - a ? result - (mod - a) : result + a;
        b >>= 1;
        if (b != 0)
            a = a >= mod - a ? a - (mod - a) : a + a;
    }
    return result;
}

typedef struct {
    size_t n;
    size_t e;
    size_t *in_offsets;
    int *in_sources;
    size_t *out_degree;
} PRGraph;

typedef struct {
    size_t n;
    size_t e;
    int demo;
    int emit_ranks;
    int graph_kind;
} PRCase;

static int pr_graph_allocate(PRGraph *g, size_t n, size_t e)
{
    if (n == 0 || n > (size_t)INT32_MAX || n == SIZE_MAX ||
        n + 1 > SIZE_MAX / sizeof(*g->in_offsets) ||
        e > SIZE_MAX / sizeof(*g->in_sources) ||
        n > SIZE_MAX / sizeof(*g->out_degree))
        return 0;
    g->n = n;
    g->e = e;
    g->in_offsets = (size_t *)calloc(n + 1, sizeof(*g->in_offsets));
    g->in_sources = (int *)malloc((e == 0 ? 1 : e) * sizeof(*g->in_sources));
    g->out_degree = (size_t *)calloc(n, sizeof(*g->out_degree));
    if (g->in_offsets == NULL || g->in_sources == NULL || g->out_degree == NULL) {
        free(g->in_offsets);
        free(g->in_sources);
        free(g->out_degree);
        memset(g, 0, sizeof(*g));
        return 0;
    }
    return 1;
}

/* A fixed affine permutation of all possible directed pairs makes generation
 * reproducible and collision-free. Self loops are skipped; pairs are unique. */
static int pr_graph_generate(PRGraph *g, size_t n, size_t wanted_edges)
{
    size_t made = 0;
```

```
    size_t k = 0;
    const uint64_t domain = (uint64_t)n * (uint64_t)n;
    const uint64_t multiplier = (uint64_t)n + 1;

    if (n == 0 || n > (size_t)INT32_MAX ||
        wanted_edges > (uint64_t)n * (uint64_t)(n - 1))
        return 0;
    if (!pr_graph_allocate(g, n, wanted_edges))
        return 0;

    while (made < wanted_edges) {
        uint64_t pair = pr_mul_mod(multiplier, (uint64_t)k, domain);
        const uint64_t offset = 17U % domain;
        pair = pair >= domain - offset ? pair - (domain - offset) : pair + offset;
        const size_t src = (size_t)(pair / n);
        const size_t dst = (size_t)(pair % n);
        ++k;
        if (src == dst)
            continue;
        ++g->in_offsets[dst + 1];
        ++g->out_degree[src];
        g->in_sources[made++] = (int)src;
    }

    for (size_t v = 0; v < n; ++v)
        g->in_offsets[v + 1] += g->in_offsets[v];

    /* Fill CSR sources in destination order. Regenerate the same pair stream. */
    {
        size_t *cursor = (size_t *)malloc(n * sizeof(*cursor));
        if (cursor == NULL) {
            free(g->in_offsets);
            free(g->in_sources);
            free(g->out_degree);
            memset(g, 0, sizeof(*g));
            return 0;
        }
        memcpy(cursor, g->in_offsets, n * sizeof(*cursor));
        made = 0;
        k = 0;
        while (made < wanted_edges) {
            uint64_t pair = pr_mul_mod(multiplier, (uint64_t)k, domain);
            const uint64_t offset = 17U % domain;
            pair = pair >= domain - offset ? pair - (domain - offset) : pair + offset;
            const size_t src = (size_t)(pair / n);
            const size_t dst = (size_t)(pair % n);
            ++k;
            if (src != dst) {
                g->in_sources[cursor[dst]++] = (int)src;
                ++made;
            }
        }
        free(cursor);
    }
    return 1;
}

static int pr_graph_from_edges(PRGraph *g, size_t n,
                               const int (*edges)[2], size_t e)
{
    if (n == 0 || n > (size_t)INT32_MAX)
        return 0;
    for (size_t i = 0; i < e; ++i)
        if (edges[i][0] < 0 || edges[i][1] < 0 ||
            (size_t)edges[i][0] >= n || (size_t)edges[i][1] >= n)
            return 0;
    if (!pr_graph_allocate(g, n, e))
        return 0;
    for (size_t i = 0; i < e; ++i) {
        const size_t src = (size_t)edges[i][0];
        const size_t dst = (size_t)edges[i][1];
        ++g->in_offsets[dst + 1];
        ++g->out_degree[src];
    }
    for (size_t v = 0; v < n; ++v)
        g->in_offsets[v + 1] += g->in_offsets[v];
    {
        size_t *cursor = (size_t *)malloc(n * sizeof(*cursor));
        if (cursor == NULL) {
            free(g->in_offsets);
            free(g->in_sources);
            free(g->out_degree);
            memset(g, 0, sizeof(*g));
```

```
            return 0;
        }
        memcpy(cursor, g->in_offsets, n * sizeof(*cursor));
        for (size_t i = 0; i < e; ++i) {
            const size_t src = (size_t)edges[i][0];
            const size_t dst = (size_t)edges[i][1];
            g->in_sources[cursor[dst]++] = (int)src;
        }
        free(cursor);
    }
    return 1;
}

/* Six-node demo, with exact directed edges: 0->1, 0->2, 1->2, 2->0,
 * 3->0, 3->2, 4->3, and 4->5. Node 5 is dangling. */
static int pr_graph_demo(PRGraph *g)
{
    static const int edges[][2] = {
        {0, 1}, {0, 2}, {1, 2}, {2, 0},
        {3, 0}, {3, 2}, {4, 3}, {4, 5}
    };
    return pr_graph_from_edges(g, 6, edges, sizeof(edges) / sizeof(edges[0]));
}

static int pr_graph_n10(PRGraph *g)
{
    static const int edges[][2] = {
        {0, 1}, {0, 2}, {1, 2}, {2, 0}, {3, 0}, {3, 2},
        {4, 3}, {4, 5}, {6, 7}, {7, 8}, {8, 6}
    };
    return pr_graph_from_edges(g, 10, edges, sizeof(edges) / sizeof(edges[0]));
}

/* Four-node examples transcribed from the supplied PageRank PDF. */
static int pr_graph1(PRGraph *g)
{
    static const int edges[][2] = {
        {0, 1}, {0, 2}, {1, 3}, {2, 3}, {3, 0}
    };
    return pr_graph_from_edges(g, 4, edges, sizeof(edges) / sizeof(edges[0]));
}

static int pr_graph2(PRGraph *g)
{
    static const int edges[][2] = {
        {0, 1}, {1, 2}, {2, 0}, {1, 3}
    };
    return pr_graph_from_edges(g, 4, edges, sizeof(edges) / sizeof(edges[0]));
}

static int pr_graph_build(PRGraph *g, const PRCase *c)
{
    switch (c->graph_kind) {
    case 1: return pr_graph_demo(g);
    case 2: return pr_graph1(g);
    case 3: return pr_graph2(g);
    case 4: return pr_graph_n10(g);
    default: return pr_graph_generate(g, c->n, c->e);
    }
}

static int pr_graph_valid(const PRGraph *g)
{
    if (g->n == 0 || g->n > (size_t)INT32_MAX || g->in_offsets == NULL ||
        g->in_sources == NULL || g->out_degree == NULL ||
        g->in_offsets[0] != 0 || g->in_offsets[g->n] != g->e)
        return 0;
    for (size_t v = 0; v < g->n; ++v)
        if (g->in_offsets[v] > g->in_offsets[v + 1] ||
            g->in_offsets[v + 1] > g->e)
            return 0;
    for (size_t p = 0; p < g->e; ++p)
        if (g->in_sources[p] < 0 || (size_t)g->in_sources[p] >= g->n ||
            g->out_degree[g->in_sources[p]] == 0)
            return 0;
    return 1;
}

static void pr_graph_free(PRGraph *g)
{
    free(g->in_offsets);
    free(g->in_sources);
    free(g->out_degree);
```

```
    memset(g, 0, sizeof(*g));
}

static int pr_parse_case(int argc, char **argv, PRCase *c)
{
    int selector_seen = 0;
    c->demo = 0;
    c->emit_ranks = 0;
    c->graph_kind = 0;
    c->n = 1000;
    c->e = 5994;
    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "--demo") == 0) {
            if (selector_seen++) return 0;
            c->demo = 1;
            c->graph_kind = 1;
        } else if (strcmp(argv[i], "--graph1") == 0) {
            if (selector_seen++) return 0;
            c->demo = 0;
            c->graph_kind = 2;
        } else if (strcmp(argv[i], "--graph2") == 0) {
            if (selector_seen++) return 0;
            c->demo = 0;
            c->graph_kind = 3;
        } else if (strcmp(argv[i], "--emit-ranks") == 0) {
            c->emit_ranks = 1;
        } else if (strcmp(argv[i], "--case") == 0 && i + 1 < argc) {
            if (selector_seen++) return 0;
            const char *name = argv[++i];
            if (strcmp(name, "N1") == 0) {
                c->n = 1; c->e = 0;
            } else if (strcmp(name, "N10") == 0) {
                c->n = 10; c->e = 11; c->graph_kind = 4;
            } else if (strcmp(name, "N100") == 0) {
                c->n = 100; c->e = 594;
            } else if (strcmp(name, "N1000") == 0 || strcmp(name, "small") == 0) {
                c->n = 1000; c->e = 5994;
            } else if (strcmp(name, "N10000") == 0 || strcmp(name, "medium") == 0) {
                c->n = 10000; c->e = 59995;
            } else if (strcmp(name, "N100000") == 0 || strcmp(name, "large") == 0) {
                c->n = 100000; c->e = 599992;
            } else {
                return 0;
            }
        } else {
            return 0;
        }
    }
    return 1;
}

static void pr_usage(const char *program)
{
    fprintf(stderr, "Usage: %s [--demo | --graph1 | --graph2 | --case
N1|N10|N100|N1000|N10000|N100000|small|medium|large] [--emit-ranks]\n", program);
}

static double *pr_initial_ranks(size_t n)
{
    if (n == 0 || n > SIZE_MAX / sizeof(double))
        return NULL;
    double *ranks = (double *)malloc(n * sizeof(*ranks));
    if (ranks != NULL)
        for (size_t i = 0; i < n; ++i)
            ranks[i] = 1.0 / (double)n;
    return ranks;
}

static double *pr_rank_vector(size_t n)
{
    return n == 0 || n > SIZE_MAX / sizeof(double) ? NULL :
           (double *)malloc(n * sizeof(double));
}

static void pr_print_result(const PRGraph *g, size_t iterations,
                            double elapsed, int success, double final_error,
                            int emit_ranks, const double *ranks)
{
    printf("%zu,%zu,%zu,%.9f,%s,%.17g\n", g->n, g->e, iterations, elapsed,
           success ? "success" : "max_iterations", final_error);
    if (g->n <= 6 && !emit_ranks) {
        printf("ranks");
        for (size_t i = 0; i < g->n; ++i)
```

```
            printf(",%.12f", ranks[i]);
        putchar('\n');
    } else if (emit_ranks) {
        printf("ranks");
        for (size_t i = 0; i < g->n; ++i)
            printf(",%.17g", ranks[i]);
        putchar('\n');
    }
}

#endif
```

Recorded MPI N10 output

Mohammed Ashar Husain | Roll No. 2024bcs0137

Run: attempt 1; MPI processes: 2

Recorded command:

```
mpirun -np 2 /home/ash/Documents/College/Sem5/PADC/PageRank/outputs/bin/mpi_pagerank --case N10 --emit-
ranks
```

Both the captured result row and all ten actual rank values are shown. Result and timing below are copied from captured stdout; identity is not inserted into program output.

```
10,11,25,0.000069972,success,5.6465066086697213e-07
ranks,0.21036722214068665,0.10830075012743358,0.21179967062274907,0.026924893717531297,0.018894662257
914793,0.026924893717531297,0.12596441505274619,0.12596441505274619,0.12596441505274619,0.01889466225
7914793
```

Recorded MPI N1000 output

Mohammed Ashar Husain | Roll No. 2024bcs0137

Run: attempt 1; MPI processes: 2

Recorded command:

```
mpirun -np 2 /home/ash/Documents/College/Sem5/PADC/PageRank/outputs/bin/mpi_pagerank --case N1000 --emit-ranks
```

Only the captured result row is shown; the logged rank vector is intentionally omitted to avoid a large output listing. Result and timing below are copied from captured stdout; identity is not inserted into program output.

```
1000,5994,43,0.000249514,success,8.8181541896921109e-07
```

Recorded MPI N100000 output

Mohammed Ashar Husain | Roll No. 2024bcs0137

Run: attempt 1; MPI processes: 2

Recorded command:

```
mpirun -np 2 /home/ash/Documents/College/Sem5/PADC/PageRank/outputs/bin/mpi_pagerank --case N100000 --emit-ranks
```

Only the captured result row is shown; the logged rank vector is intentionally omitted to avoid a large output listing. Result and timing below are copied from captured stdout; identity is not inserted into program output.

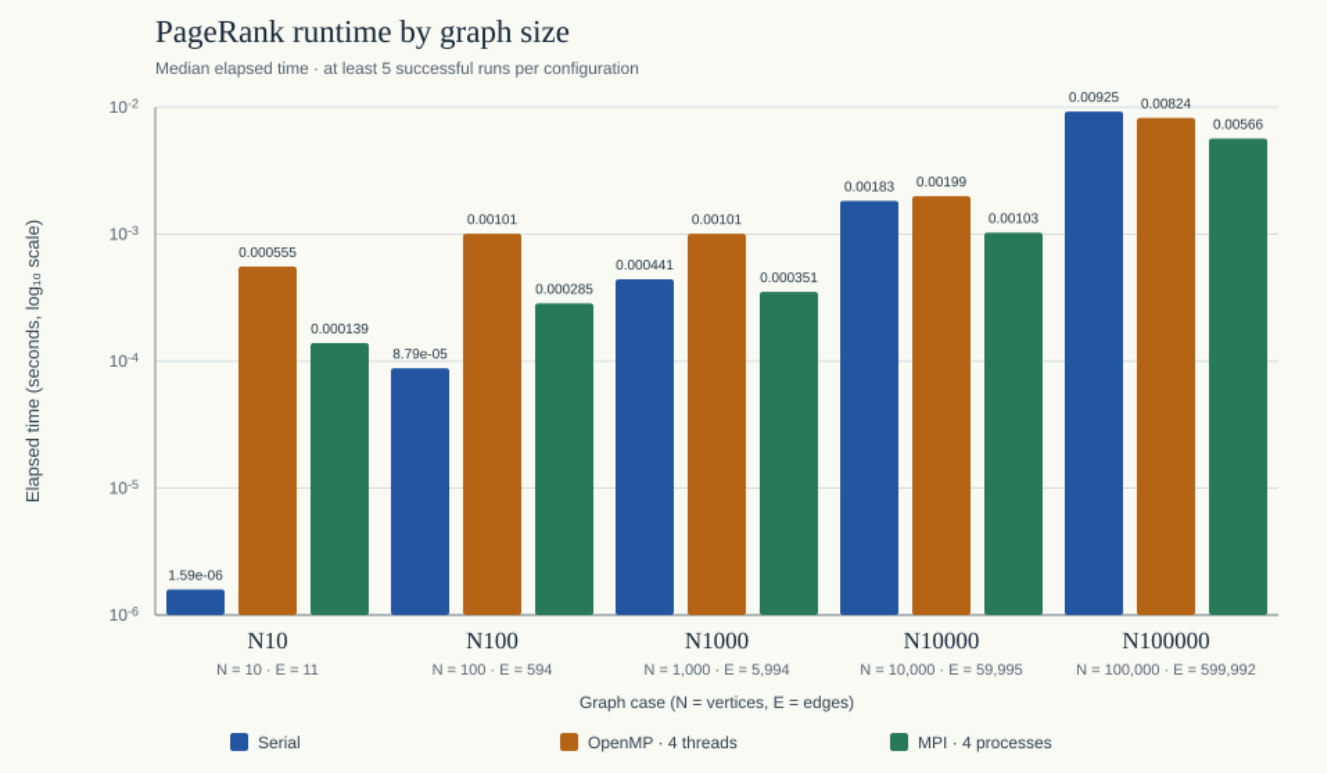
```
100000,599992,16,0.006787760,success,9.6722143367025459e-07
```

Q11 — Five-size execution-time comparison

Mohammed Ashar Husain | Roll No. 2024bcs0137

Medians are read from summary rows in outputs/comparison.csv. Each summary is the median of five successful timed runs for that configuration (one warmup is excluded).

Case	N	E	Serial (median seconds)	OpenMP / 4 (median seconds)	MPI / 2 (median seconds)	MPI / 4 (median seconds)
N10	10	11	1.587e-06	0.000554799	6.4743e-05	0.000138717
N100	100	594	8.7861e-05	0.001007639	0.000116308	0.000284884
N1000	1,000	5,994	0.000441013	0.001008721	0.000256088	0.000350771
N10000	10,000	59,995	0.001829857	0.00199082	0.001254603	0.001029242
N100000	100,000	599,992	0.009252197	0.008243165	0.006543447	0.005660108



Current project graph from outputs/comparison.png (includes serial, OpenMP/4 and MPI/4 series).